Arithmetic Logic Unit (ALU) Design Using Verilog HDL

RTL Design, Simulation and Verification Report

By

Priya Nageswari Karanam

Table of Contents

1 Abstract

This project presents the implementation of a 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL. The designed ALU performs multiple arithmetic and logical operations including addition, subtraction, AND, OR, XOR, shift operations, and complement operation based on a control selection signal.

The design is developed using combinational RTL architecture and verified through simulation using Vivado and ModelSim. A dedicated testbench is created to analyze the functional behavior of the ALU under different input conditions. Simulation waveforms confirm correct operation execution and carry generation during arithmetic computations.

The project provides practical exposure to combinational circuit design, RTL development, Verilog operators, operation selection mechanisms, and simulation-based functional verification techniques.

2 Introduction

Digital systems require hardware units capable of performing mathematical and logical computations efficiently. The Arithmetic Logic Unit (ALU) is a core computational component used in processors, controllers, and embedded systems to execute such operations on binary data.

This project focuses on designing a 4-bit ALU capable of performing different operations through a selection-based control mechanism. Depending on the control input, the ALU executes arithmetic operations such as addition and subtraction, along with logical operations including AND, OR, XOR, shifting, and bitwise inversion.

The entire functionality is implemented using Verilog HDL through combinational logic modeling. Simulation and verification are carried out using Vivado and ModelSim to analyze output behavior for multiple input combinations. The project also introduces the use of case statements, operator-based hardware modeling, and waveform analysis techniques in RTL design.

Through this work, a foundational understanding of combinational digital design and hardware description language based system modeling is developed.

3 Objective

- To design a 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL.
- To perform arithmetic operations such as addition and subtraction using combinational logic.
- To implement logical operations including AND, OR, XOR, and NOT operations.
- To perform bit manipulation using left shift and right shift operations.
- To develop operation selection logic using a control signal and case statement implementation.
- To generate carry output during arithmetic computations.
- To verify the functionality of the ALU using Vivado and ModelSim simulations.
- To analyze output behavior using waveform-based verification techniques.

4 Design Methodology

The 4-bit ALU is designed using combinational digital logic in Verilog HDL. The design accepts two 4-bit binary inputs, A and B, along with a 3-bit selection signal sel. Based on the value of the selection signal, the ALU performs different arithmetic and logical operations and produces the corresponding output.

The operation selection mechanism is implemented using a case statement inside an always @(*) block. Since the ALU is a combinational circuit, the output changes immediately whenever there is a change in the input signals or selection signal.

The arithmetic section of the ALU performs addition and subtraction operations. Logical operations including AND, OR, XOR, and NOT are implemented using Verilog bitwise operators. Shift operations are performed using left shift and right shift operators. A carry output is also generated during arithmetic operations to indicate overflow conditions.

The functionality of the ALU is verified using simulation in Vivado and ModelSim. Multiple input combinations are applied through a dedicated Verilog testbench to validate operation selection, carry generation, and output correctness.

ALU Operations

Selection Signal	Operation
000	Addition
001	Subtraction
010	AND
011	OR
100	XOR
101	Left Shift
110	Right Shift
111	NOT

Table 1: Operations Performed by the ALU

5 Verilog Code

```
module ALU(
    input [3:0] A, input [3:0] B, input [2:0] sel,
    output reg [3:0] out, output reg carry);
    always @(*) begin
        carry = 0;
        case(sel)
            3'b000 : {carry,out} = A + B;
            3'b001 : {carry,out} = A - B;
            3'b010 : out = A & B;
            3'b011 : out = A | B;
            3'b100 : out = A ^ B;
            3'b101 : out = A << 1;
```

```
            3'b110 : out = A >> 1;
            3'b111 : out = ~A;
            default : out = 4'b0000;
        endcase
    end
endmodule
```

6 Testbench Code

```
module ALU_tb;
    reg [3:0] A;  reg [3:0] B; reg [2:0] sel;
    wire [3:0] out; wire carry;

    ALU dut(A, B, sel, out, carry);

    initial begin
        $monitor("sel=%b A=%b B=%b out=%b carry=%b", sel, A, B, out, carry);

    // Test Case 1
        A = 4'b1111; B = 4'b0001;
        sel = 3'b000; #10;
        sel = 3'b001; #10;
        sel = 3'b010; #10;

    // Test Case 2
        A = 4'b1010; B = 4'b0101;
        sel = 3'b011; #10;
        sel = 3'b100; #10;

    // Test Case 3
        A = 4'b1100; B = 4'b0011;
        sel = 3'b101; #10;
        sel = 3'b110; #10;

    // Test Case 4
        A = 4'b1111; B = 4'b1111;
        sel = 3'b111; #10;
        $stop;
    end
endmodule
```

7 Code Explanation

The ALU module is designed with two 4-bit inputs A and B, a 3-bit selection signal sel, a 4-bit output out, and a carry output signal. The selection signal determines which operation is executed inside the ALU.

The design uses an always @(*) block to implement combinational logic behavior. This ensures that the output updates immediately whenever any input signal changes.

Inside the always block, the carry signal is initially assigned to zero before executing the selected operation. This prevents unintended carry retention during non-arithmetic operations.

A case statement is used to implement operation selection. Depending on the value of sel, the ALU performs different operations as listed below:

- sel = 000 performs addition of A and B.
- sel = 001 performs subtraction of A and B.
- sel = 010 performs bitwise AND operation.
- sel = 011 performs bitwise OR operation.
- sel = 100 performs bitwise XOR operation.
- sel = 101 performs left shift operation on A.
- sel = 110 performs right shift operation on A.
- sel = 111 performs bitwise complement operation on A.

For arithmetic operations, concatenation is used in the form carry,out to store both the carry bit and output result simultaneously.

The testbench module is used to apply multiple input combinations and verify the functionality of the ALU. Different selection values are applied sequentially to analyze output behavior and carry generation using simulation waveforms.

8 RTL Schematic

The RTL schematic generated in Vivado represents the internal combinational structure of the ALU design. The schematic illustrates the connection between input signals, operation selection logic, arithmetic blocks, logical operation blocks, shift operation logic, and output generation circuitry.

The design mainly consists of combinational operator blocks controlled through the selection signal sel. Depending on the selected operation, the corresponding result is routed to the output through operation selection logic.

The RTL view also shows the implementation of carry generation during arithmetic operations and the overall data flow between inputs and outputs within the ALU architecture.

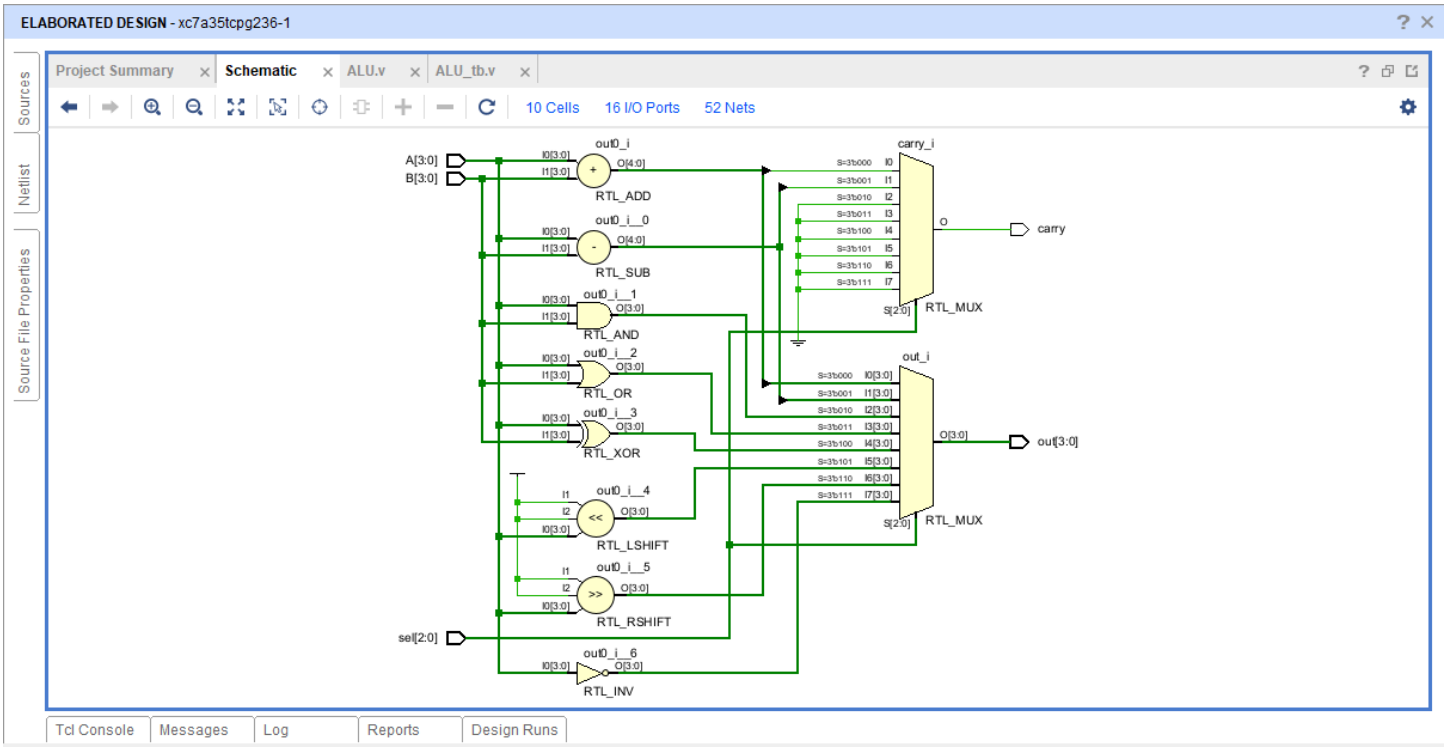

Figure: RTL Schematic of 4-Bit ALU

9 Simulation Results

The functionality of the 4-bit ALU is verified using simulation in Vivado and ModelSim. Multiple input combinations are applied through the Verilog testbench to analyze arithmetic operations, logical operations, shift operations, and carry generation.

The simulation waveforms confirm correct operation selection based on the value of the selection signal sel. Arithmetic operations such as addition and subtraction generate appropriate outputs along with carry indication during overflow conditions. Logical operations including AND, OR, XOR, and NOT are also verified successfully through waveform analysis.

Shift operations are validated using left shift and right shift conditions. The output waveforms demonstrate correct combinational behavior of the ALU for different input combinations and selection values.

Vivado Simulation Output

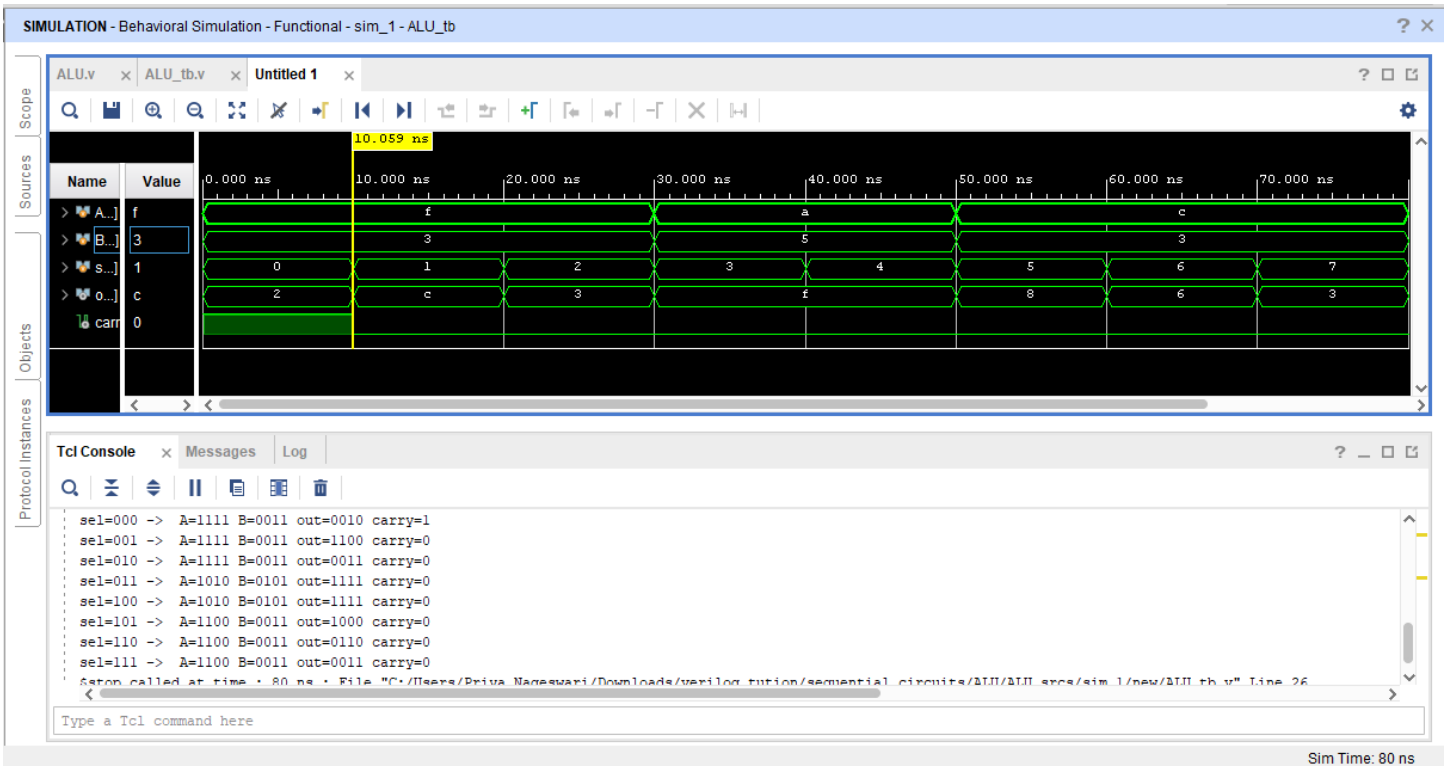

Figure: Vivado Simulation Waveform of 4-Bit ALU

ModelSim Simulation Output

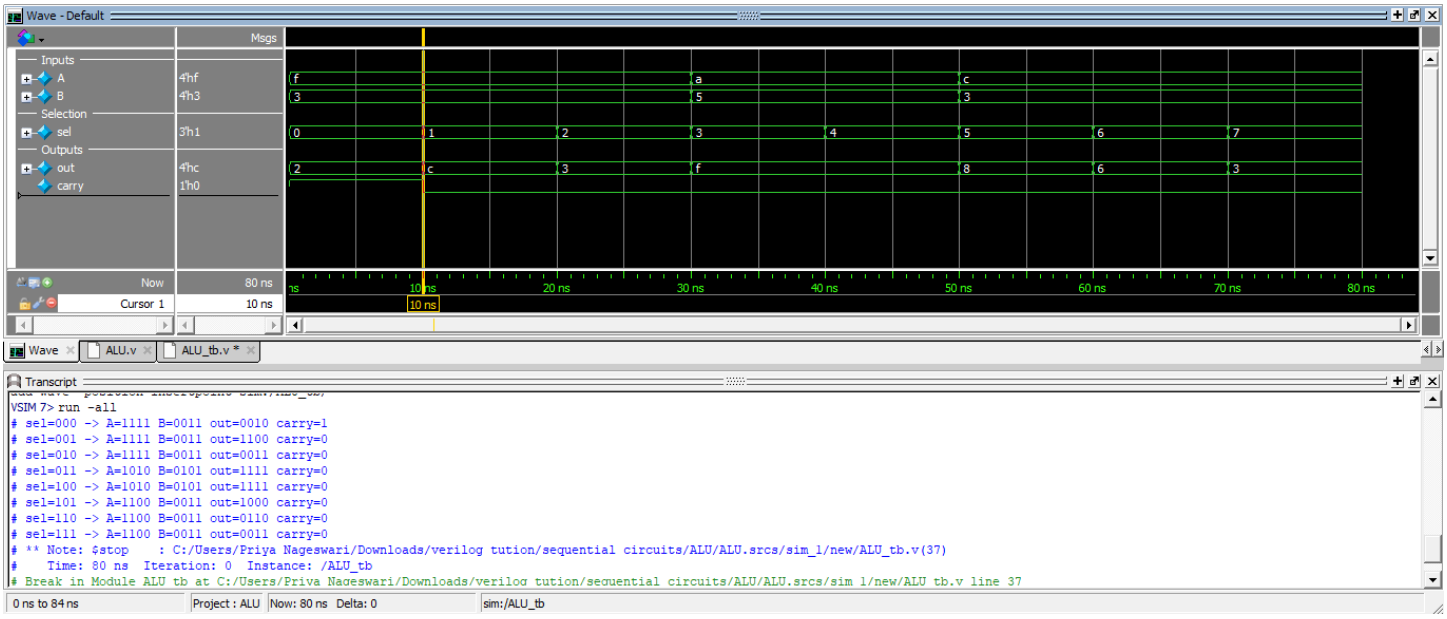

Figure: ModelSim Simulation Waveform of 4-Bit ALU

10 Future Scope

Additional Operations

- Multiplication and division operations can be implemented to improve arithmetic capability of the ALU.
- Increment, decrement, and comparison operations can also be added for advanced computation functionality.

Higher Bit Architectures

- The current 4-bit ALU can be extended to 8-bit, 16-bit, or 32-bit architectures.
- Wider architectures are commonly used in processors and advanced digital systems.

Status Flags

- Additional flags such as zero flag, overflow flag, and sign flag can be incorporated.
- These flags help in improving processor-level decision making and arithmetic analysis.

FPGA Implementation

- The ALU design can be implemented on FPGA hardware for real-time operation.
- FPGA implementation helps in understanding hardware deployment and resource utilization.

11 Conclusion

The 4-bit Arithmetic Logic Unit (ALU) was successfully designed and simulated using Verilog HDL. The project demonstrated the implementation of combinational digital logic for performing arithmetic and logical operations based on a selection control signal.

The functionality of the ALU was verified using Vivado and ModelSim simulation tools along with a dedicated Verilog testbench. The simulation results confirmed correct operation selection, output generation, carry functionality, and combinational behavior of the design.

This project provided practical understanding of combinational circuit design, Verilog operators, RTL modeling, operation selection logic, and simulation-based verification techniques using Verilog HDL.